

CS8165.001 Complete Lecture Readers

Readable commentary for each lecture in the course's primary language. This document is designed to make the raw slides human-readable while keeping links back to the extracted source chunks.

Contents

Lecture 01 - Introduction

Lecture 02 - Rendering Pipeline

Lecture 03 - Geometric Transformations

Lecture 04 - Geometric Projection

Lecture 05 - Clipping

Lecture 06 - Rasterization

Lecture 07 - Visibility Determination

Lecture 08 - Local Illumination

Lecture 09 - Texturing

Lecture 10 - Shadows

Lecture 01 - Introduction

This lecture is the orientation layer for the whole course. It explains why interactive computer graphics is a pipeline problem: a scene must be represented, processed under time constraints, sampled into an image, and displayed fast enough for interaction.

Source chunk	course_text_parts/03_lectures/01_introduction.txt
Extracted slide pages	51
Reader purpose	Commentary and causal explanation for human reading

How to read this lecture

- Read the section commentary before looking at the raw slide bullets.
- Use the mental model as the short version you should remember.
- Answer the check question without notes before continuing.

Slide walkthrough

Each row gives a short reading comment for one extracted slide page. Use it to turn sparse slide bullets into a human-readable path through the lecture.

Page	Slide title	How to read it
1	Untitled slide	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
2	Course Learning Outcomes	Read this as organization and relevance information: it tells you how lecture theory, exercises, and exam preparation connect.
3	Real-Time Rendering Contest Results 1/3	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
4	Real-Time Rendering Contest Results 2/3	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
5	Real-Time Rendering Contest Results 3/3	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
6	1.1 Course Organization	Read this as organization and relevance information: it tells you how lecture theory, exercises, and exam preparation connect.
7	Teachers	Read this as organization and relevance information: it tells you how lecture theory, exercises, and exam preparation connect.
8	Course Format	Read this as organization and relevance information: it tells you how lecture theory, exercises, and exam preparation connect.

Page	Slide title	How to read it
9	Course Format	Read this as organization and relevance information: it tells you how lecture theory, exercises, and exam preparation connect.
10	Exercises	Read this as organization and relevance information: it tells you how lecture theory, exercises, and exam preparation connect.
11	Successful Course Completion	Read this as organization and relevance information: it tells you how lecture theory, exercises, and exam preparation connect.
12	Course Overview	Read this as organization and relevance information: it tells you how lecture theory, exercises, and exam preparation connect.
13	Literature	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
14	Relevant Chapters and Link to Lecture Course	Read this as organization and relevance information: it tells you how lecture theory, exercises, and exam preparation connect.
15	(1.1 Course Organization)	Read this as organization and relevance information: it tells you how lecture theory, exercises, and exam preparation connect.
16	1.2 Computer Graphics Overview	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
17	Computer Graphics History	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
18	Computer Graphics History	Read this as boundary logic. Identify what is inside, what is outside, and when new intersection vertices are created.
19	Computer Graphics History	Read this as boundary logic. Identify what is inside, what is outside, and when new intersection vertices are created.
20	Computer Graphics History	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
21	Computer Graphics History	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
22	Related Disciplines	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
23	Applications	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?

Page	Slide title	How to read it
24	1.3 Pixel-based Representations	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
25	Raster Images	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
26	Pixel	Read this as an occlusion decision. Decide whether the method works in object space, image space, or per-fragment depth space.
27	Color	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
28	LCD Raster Screen	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
29	Raster Image Sizes	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
30	1.4 3D Models	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
31	3D Model Types	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
32	Implicit Surfaces	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
33	Quadrics	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
34	Object Representation	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
35	Polygonal Models 1/3	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
36	Polygonal Models 1/4	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
37	Polygonal Models 2/4	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
38	Polygonal Models 3/4	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.

Page	Slide title	How to read it
39	Polygonal Models 4/4	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
40	Volumetric Models	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
41	Model Generation	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
42	Coordinate Systems	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
43	Rendering	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
44	1.5 Algorithmic Paradigms	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
45	Photorealistic Rendering	Read this as an occlusion decision. Decide whether the method works in object space, image space, or per-fragment depth space.
46	Non-Photorealistic Rendering (NPR)	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
47	Image-Based Algorithms	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
48	Object-Based Algorithms	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
49	are assigned	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
50	Literature and other sources used in this chapter	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
51	Peters 2016. (Chapters 1, 3 & 21)	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.

01.1 Course Organization

Source location: Slides around 6-15

Commentary

The organizational slides matter because they explain how the theory and programming parts fit together. The exercises are not separate from the lecture; they are the practical version of the same pipeline ideas. When the course says that slides are generally not self-explanatory, that is a warning: the bullet points are anchors, not a textbook.

Treat every exercise sheet as an applied checkpoint. Rendering pipeline, primitive types, shaders, transformations, projection, clipping, and rasterization are introduced in the lectures and then turned into OpenGL tasks. If a topic appears both in a lecture and an exercise, it is more likely to be exam relevant.

Mental model

Lecture slides give the vocabulary; exercises force you to connect that vocabulary to code and visible output.

Check yourself

Can you list the five exercise themes and connect each one to a later lecture chapter?

01.2 Computer Graphics Overview

Source location: Section 1.2

Commentary

Computer graphics is about generating images from descriptions. The description can be pixel-based, object-based, physically motivated, artistic, or algorithmic. The common thread is that the computer must decide what color belongs at image samples.

Interactive graphics adds a time constraint. The image is not computed once and admired; it must respond to camera motion, user input, animation, and changing state. That is why real-time rendering often uses approximations and specialized GPU stages.

Mental model

Graphics is controlled image generation. Interactive graphics is controlled image generation under a strict time budget.

Check yourself

What changes when a renderer must be interactive rather than offline?

01.3 Pixel-Based Representations

Source location: Section 1.3

Commentary

A pixel-based representation stores an image directly as samples on a grid. It is easy to display and edit locally, but it does not know the underlying 3D scene. If you zoom into a raster image, you reveal the sampling grid, not more geometry.

This is the first place where sampling becomes important. Resolution, color depth, aliasing, and frame time are not cosmetic details. They decide what information can be represented and how much data must be processed per second.

Mental model

A raster image is already the answer. It contains color samples, not the geometric reasons behind them.

Check yourself

Why can a pixel image be easy to display but hard to reinterpret as a 3D scene?

01.4 3D Models

Source location: Section 1.4

Commentary

A 3D model is a structured description of objects before they become pixels. It may contain vertices, primitives, topology, normals, materials, textures, and transformations. Unlike a pixel image, it can be viewed from different camera positions.

The course mostly follows object-based rendering: start from model data, transform it, project it, rasterize it, shade it, test visibility, and finally update pixels. Later chapters explain these steps one by one.

Mental model

A model is not an image. It is a cause from which many possible images can be generated.

Check yourself

Which model attributes can influence final color besides vertex positions?

01.5 Algorithmic Paradigms

Source location: Section 1.5

Commentary

The lecture contrasts ways to generate images. Rasterization works from objects toward pixels and is the dominant real-time pipeline. Ray-based methods work from image samples into the scene and are powerful for visibility and lighting but can be more expensive.

Do not memorize these paradigms as names only. Ask what direction information flows. Does the algorithm start from geometry and find covered pixels, or from pixels/rays and find visible geometry? That direction explains the strengths and weaknesses.

Mental model

Rasterization asks, 'Which samples does this primitive cover?' Ray casting asks, 'What does this sample see?'

Check yourself

Which paradigm fits OpenGL's standard real-time pipeline most directly?

End-of-lecture summary

If you remember only one thing from Lecture 01, remember this: This lecture is the orientation layer for the whole course. It explains why interactive computer graphics is a pipeline problem: a scene must be represented, processed under time constraints, sampled into an image, and displayed fast enough for interaction.

Before moving on, you should be able to explain the lecture without reading slide bullets aloud. Use the vocabulary from the slides, but add causal links: why a step exists, what data it consumes, what it produces, and what can go wrong.

Lecture 02 - Rendering Pipeline

This lecture turns the vague idea of rendering into a concrete sequence. It is the most important debugging map in the course: if an image is wrong, the pipeline tells you where the error could have entered.

Source chunk	course_text_parts/03_lectures/02_rendering-pipeline.txt
Extracted slide pages	94
Reader purpose	Commentary and causal explanation for human reading

How to read this lecture

- Read the section commentary before looking at the raw slide bullets.
- Use the mental model as the short version you should remember.
- Answer the check question without notes before continuing.

Slide walkthrough

Each row gives a short reading comment for one extracted slide page. Use it to turn sparse slide bullets into a human-readable path through the lecture.

Page	Slide title	How to read it
1	Untitled slide	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
2	Object-based Rendering	Read this as the main data-flow story: scene/model data is processed step by step until only valid framebuffer updates remain.
3	2.1 Rendering Process	Read this as the main data-flow story: scene/model data is processed step by step until only valid framebuffer updates remain.
4	2.1 Rendering Process	Read this as the main data-flow story: scene/model data is processed step by step until only valid framebuffer updates remain.
5	Geometry-based Rendering 1/2	Read this as the main data-flow story: scene/model data is processed step by step until only valid framebuffer updates remain.
6	Geometry-based Rendering 2/2	Read this as the main data-flow story: scene/model data is processed step by step until only valid framebuffer updates remain.
7	Application Stage	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
8	Geometry Stage	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.

Page	Slide title	How to read it
9	Rasterization Stage	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
10	2.2 OpenGL Overview	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
11	OpenGL History	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
12	OpenGL	Read this as the main data-flow story: scene/model data is processed step by step until only valid framebuffer updates remain.
13	OpenGL vs. Vulkan	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
14	OpenGL Libraries	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
15	OpenGL Functionality	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
16	OpenGL Context	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
17	OpenGL Rendering Process	Read this as the main data-flow story: scene/model data is processed step by step until only valid framebuffer updates remain.
18	Window Creation with GLFW 1/2	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
19	Window Creation with GLFW 2/2	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
20	OpenGL Naming Conventions	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
21	2.3 OpenGL Rendering	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
22	OpenGL Rendering (until V2.1)	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
23	OpenGL Rendering (since V3.0)	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.

Page	Slide title	How to read it
24	1. Vertex Specification	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
25	2. Vertex Data Transfer	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
26	3. Memory Layout Description	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
27	4. VAO Drawing 1/2	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
28	4. VAO Drawing 2/2	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
29	5. GPU Memory Release	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
30	Rendering Initialization 1/2	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
31	Rendering Initialization 2/2	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
32	Rendering	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
33	Additional Attributes	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
34	2.4 OpenGL Rendering Pipeline	Read this as the main data-flow story: scene/model data is processed step by step until only valid framebuffer updates remain.
35	The Rendering Pipeline 1/2	Read this as the main data-flow story: scene/model data is processed step by step until only valid framebuffer updates remain.
36	The Rendering Pipeline 2/2	Read this as the main data-flow story: scene/model data is processed step by step until only valid framebuffer updates remain.
37	Shader Programming – History 1/2	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
38	Shader Programming – History 2/2	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.

Page	Slide title	How to read it
39	Assembler Shader	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
40	High-Level Shading Languages 1/2	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
41	High-Level Shading Languages 2/2	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
42	Shader Types	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
43	Vertex Shader Concepts	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
44	Vertex Shader Examples 1/2	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
45	Vertex Shader Examples 2/2	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
46	Vertex Shader Coordinate Systems 1/2	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
47	Vertex Shader Coordinate Systems 2/2	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
48	Local Space	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
49	World Space	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
50	View Space	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
51	Clip Space	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
52	Screen Space	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
53	Coordinate Systems	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.

Page	Slide title	How to read it
54	Vertex Shader Variables	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
55	Fixed Function Vertex Postprocessing	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
56	Fixed Function Rasterization	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
57	Fragment Shader Concept 1/2	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
58	Fragment Shader Concept 2/2	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
59	Shader Example	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
60	GLSL Shader Functions	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
61	Shader Usage	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
62	Binding Shaders	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
63	Setting Uniforms	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
64	Shader Performance	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
65	Shader Development Environments 1/2	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
66	Shader Development Environments 2/2	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
67	2.5 Fragment Tests and Operations	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
68	Fragment Tests and Operations 1/2	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?

Page	Slide title	How to read it
69	Fragment Tests and Operations 2/2	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
70	Alpha Test	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
71	Stencil Test	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
72	Depth Test	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
73	Image Composition and Mixture	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
74	Blending Application	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
75	Blending Process 1/2	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
76	Blending Process 2/2	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
77	Applying Blending	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
78	Blending Examples	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
79	2.6 Framebuffer	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
80	Viewport	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
81	Framebuffer	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
82	OpenGL Framebuffers 1/3	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
83	OpenGL Framebuffers 2/3	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.

Page	Slide title	How to read it
84	OpenGL Framebuffers 3/3	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
85	Double Buffering	Read this as the main data-flow story: scene/model data is processed step by step until only valid framebuffer updates remain.
86	2.7 Pixel-based Rendering	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
87	Pixel-based Rendering Operations	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
88	Image Operations	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
89	Write Pixels into Framebuffer	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
90	Read Pixels from Framebuffer	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
91	Formats & Data Types	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
92	coordinate systems in the geometry stage	Read this as the main data-flow story: scene/model data is processed step by step until only valid framebuffer updates remain.
93	Literature and other sources used in this chapter	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
94	Guide: The Official Guide to Learning OpenGL	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.

02.1 Rendering Process

Source location: Section 2.1

Commentary

Object-based rendering starts with scene objects represented as primitives. The pipeline transforms vertices, assembles primitives, rasterizes them into fragments, and lets only some fragments become pixels. The important detail is that the representation changes at every stage.

The application stage prepares data and state. The geometry stage handles positions and primitives. The rasterization stage creates fragments and applies fragment operations. This separation helps you explain both theory questions and black-screen OpenGL bugs.

Mental model

Vertices become primitives, primitives become fragments, and fragments compete to become pixel updates.

Check yourself

What is the difference between a fragment and a final pixel?

02.2 OpenGL Overview

Source location: Section 2.2

Commentary

OpenGL is an API for controlling a graphics pipeline. It is not a renderer by itself in the sense of a single command that understands your scene. You create a context, provide buffers, set state, compile shaders, bind objects, and issue draw calls.

Modern OpenGL is stateful and shader-based. Many errors happen because the programmer assumes that a later call carries more information than it really does. The GPU uses the currently bound objects and current state at draw time.

Mental model

OpenGL draw calls consume the state machine as it exists right now.

Check yourself

Why can binding the wrong vertex array or shader program produce a valid draw call with wrong output?

02.3 OpenGL Rendering

Source location: Section 2.3

Commentary

Older OpenGL exposed a fixed-function style where many transformations and lighting choices were configured through predefined calls. Modern OpenGL expects you to write shader programs and explicitly manage data flow.

This is not just historical trivia. It explains why shader code, vertex attributes, uniforms, and buffer objects are central in the exercises. If the shader expects an attribute and the vertex layout does not provide it, the pipeline has no magic fallback.

Mental model

Modern OpenGL makes the data path explicit; that gives control but also creates responsibility.

Check yourself

Which data usually goes into vertex attributes, and which data usually goes into uniforms?

02.4 OpenGL Rendering Pipeline

Source location: Section 2.4

Commentary

The vertex shader runs per vertex and usually outputs clip-space position plus attributes for later interpolation. Primitive assembly groups vertices into points, lines, or triangles. Clipping handles geometry outside the view volume. Rasterization creates fragments.

The fragment shader computes candidate fragment output, often using interpolated attributes, uniforms, and textures. After that, tests and operations such as depth testing, stencil testing, blending, and masking decide what reaches the framebuffer.

Mental model

Programmable shaders compute values; fixed pipeline stages decide coverage, interpolation, tests, and output rules.

Check yourself

If your geometry appears in wireframe but colors are wrong, which stages become suspicious?

02.5 Fragment Tests, Framebuffer, and Pixel-Based Rendering

Source location: Sections 2.5-2.7

Commentary

Fragment tests are the gatekeepers after shading. A fragment can have a perfectly valid color and still fail because depth, stencil, scissor, masking, or blending state prevents a visible update. This is why final pixels are a subset of generated fragments.

The framebuffer stores the final render targets: color buffers, depth buffers, stencil buffers, or custom attachments. Pixel-based rendering starts from image data instead of geometric primitives, which is useful for image processing but conceptually different from the object-to-pixel pipeline.

Mental model

The framebuffer is the destination; fragment operations are the rules for writing into it.

Check yourself

Why can two fragments with different colors produce only one visible pixel color?

End-of-lecture summary

If you remember only one thing from Lecture 02, remember this: This lecture turns the vague idea of rendering into a concrete sequence. It is the most important debugging map in the course: if an image is wrong, the pipeline tells you where the error could have entered.

Before moving on, you should be able to explain the lecture without reading slide bullets aloud. Use the vocabulary from the slides, but add causal links: why a step exists, what data it consumes, what it produces, and what can go wrong.

Lecture 03 - Geometric Transformations

This lecture explains how geometry moves between coordinate systems. Almost every later topic assumes that you can track where a point or vector currently lives.

Source chunk	course_text_parts/03_lectures/03_geometric-transformations.txt
Extracted slide pages	54
Reader purpose	Commentary and causal explanation for human reading

How to read this lecture

- Read the section commentary before looking at the raw slide bullets.
- Use the mental model as the short version you should remember.
- Answer the check question without notes before continuing.

Slide walkthrough

Each row gives a short reading comment for one extracted slide page. Use it to turn sparse slide bullets into a human-readable path through the lecture.

Page	Slide title	How to read it
1	Untitled slide	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
2	Transforming 3D Models	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
3	3.1 Mathematical Foundations	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
4	3.1 Mathematical Foundations	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
5	Vertices as Vectors	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
6	Transformations as Matrices	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
7	Matrix Vector Multiplication	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
8	3.2 Affine Transformations	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.

Page	Slide title	How to read it
9	Translations – Moving 3D Models	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
10	Translation Properties	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
11	Scaling – Resizing 3D Models	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
12	Scaling Properties	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
13	Rotation – Changing Orientation of 3D Models 1/2	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
14	Rotation – Changing Orientation of 3D Models 2/2	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
15	Rotation Properties	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
16	Mirroring – Reflecting 3D Models on a Plane	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
17	Shearing – Skewing 3D Models	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
18	3.3 Composition of Transformations	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
19	Composition of Transformations	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
20	Reference Points 1/2	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
21	Reference Points 2/2	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
22	Rotation Around Arbitrary Axis	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
23	Reminder: Vector Calculus	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.

Page	Slide title	How to read it
24	Reminder: Cross Product (= vector product)	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
25	Example – Transforming Directed Line Segment 1/7	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
26	Example – Transforming Directed Line Segment 2/7	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
27	Example – Transforming Directed Line Segment 3/7	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
28	Example – Transforming Directed Line Segment 4/7	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
29	Example – Transforming Directed Line Segment 5/7	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
30	Example – Transforming Directed Line Segment 6/7	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
31	Example – Transforming Directed Line Segment 7/7	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
32	Example – Direction of Flight Transformation 1/3	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
33	Example – Direction of Flight Transformation 2/3	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
34	Example – Direction of Flight Transformation 3/3	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
35	3.4 Coordinate System Change	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
36	Coordinate System Change	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
37	Local Coordinate Systems	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
38	Example 1 – Coordinate System Change 1/2	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.

Page	Slide title	How to read it
39	Example 1 – Coordinate System Change 2/2	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
40	Example 2 – Hierarchical Coordinate Systems 1/2	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
41	Example 2 – Hierarchical Coordinate Systems 1/2	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
42	3.5 Transformations in OpenGL	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
43	Geometric Transformations in OpenGL	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
44	OpenGL Transformation Pipeline	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
45	Example – Translation and Scaling	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
46	Matrix Specification	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
47	Example – Portico Modeling 1/5	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
48	Example – Portico Modeling 2/5	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
49	Example – Portico Modeling 3/5	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
50	Example – Portico Modeling 4/5	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
51	Example – Portico Modeling 5/5	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
52	by applying affine transformations to their vertices	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
53	Literature and other sources used in this chapter	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?

Page	Slide title	How to read it
54	(3rd Edition), Addison-Wesley 2013. (Chapters 10 & 11)	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?

03.1 Mathematical Foundations

Source location: Section 3.1

Commentary

The lecture uses vectors and matrices to express geometric operations. A vector can represent a point position, a direction, an offset, or an attribute depending on context. A matrix expresses a linear map or, with homogeneous coordinates, an affine transformation.

The central exam habit is to name the coordinate space. A point in model coordinates and a light direction in view coordinates cannot be mixed safely. Many visual bugs are coordinate-space bugs.

Mental model

A numeric vector is incomplete until you know what coordinate system it belongs to.

Check yourself

Why is a normal vector not transformed exactly like a point under all transformations?

03.2 Affine Transformations

Source location: Section 3.2

Commentary

Affine transformations include translation, rotation, scaling, shearing, and combinations of these. Homogeneous coordinates allow translation to be represented in matrix form together with other transformations.

Translation moves positions but not pure directions. Rotation changes orientation while preserving lengths if it is a proper rotation. Scaling can change lengths and can also affect normals. These distinctions matter later in lighting.

Mental model

Use 4D homogeneous coordinates so that a single matrix pipeline can move 3D points through the scene.

Check yourself

What does the homogeneous w component let a transformation matrix express?

03.3 Composition of Transformations

Source location: Section 3.3

Commentary

Composition means applying several transformations in sequence. The order matters because matrix multiplication is generally not commutative. Rotating an object around its own origin and then translating it is different from translating it and then rotating around the world origin.

When you read a formula such as $\text{projection} * \text{view} * \text{model} * \text{position}$, read it from right to left for the point: first model, then view, then projection. The resulting matrix chain is compact, but the conceptual steps remain separate.

Mental model

Matrix chains are compressed stories of movement through coordinate spaces.

Check yourself

Why can swapping model and view matrices destroy the intended camera/object relation?

03.4 Coordinate System Change

Source location: Section 3.4

Commentary

A transformation can be read actively as moving an object, or passively as changing the coordinate frame used to describe it. Both interpretations are valid, but mixing them carelessly causes sign and order errors.

The view matrix is a classic example: conceptually you position a camera, but the pipeline usually transforms the world by the inverse of the camera transform so that the camera becomes the origin of view space.

Mental model

Moving the camera one way is equivalent to moving the world the opposite way for rendering.

Check yourself

Why is the view transform often related to the inverse camera transform?

03.5 Transformations in OpenGL

Source location: Section 3.5

Commentary

Modern OpenGL does not automatically manage the old matrix stacks. You usually compute model, view, and projection matrices in application code and pass them to shaders as uniforms.

The vertex shader is where the final clip-space position is normally computed. That means a wrong matrix uniform, wrong multiplication order, or wrong convention can make geometry vanish even though buffers and draw calls are correct.

Mental model

The shader is the place where abstract transformation math becomes GPU execution.

Check yourself

Which matrix would you inspect first if an object follows the camera instead of staying in the world?

End-of-lecture summary

If you remember only one thing from Lecture 03, remember this: This lecture explains how geometry moves between coordinate systems. Almost every later topic assumes that you can track where a point or vector currently lives.

Before moving on, you should be able to explain the lecture without reading slide bullets aloud. Use the vocabulary from the slides, but add causal links: why a step exists, what data it consumes, what it produces, and what can go wrong.

Lecture 04 - Geometric Projection

This lecture explains how a 3D view becomes a 2D image. Projection is where camera geometry, homogeneous coordinates, clipping planes, depth precision, and viewport mapping meet.

Source chunk	course_text_parts/03_lectures/04_geometric-projection.txt
Extracted slide pages	61
Reader purpose	Commentary and causal explanation for human reading

How to read this lecture

- Read the section commentary before looking at the raw slide bullets.
- Use the mental model as the short version you should remember.
- Answer the check question without notes before continuing.

Slide walkthrough

Each row gives a short reading comment for one extracted slide page. Use it to turn sparse slide bullets into a human-readable path through the lecture.

Page	Slide title	How to read it
1	Untitled slide	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
2	Projecting 3D Models	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
3	4.1 Linear Perspective	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
4	4.1 Linear Perspective	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
5	Linear Perspective	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
6	Drawing by Projecting	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
7	Projection in the Arts 1/5	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
8	Projection in the Arts 2/5	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.

Page	Slide title	How to read it
9	Projection in the Arts 3/5	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
10	Projection in the Arts 4/5	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
11	Projection in the Arts 4/5	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
12	Projection in the Arts 5/5	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
13	Projection in the Arts 6/5	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
14	Projection Technics 1/3	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
15	Projection Technics 1/3	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
16	Projection Technics 2/3	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
17	Projection Technics 3/3	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
18	Projection Technics 3/3	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
19	4.2 Planar Projections	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
20	Geometric Projections	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
21	Projection Types 1/2	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
22	Projection Types 2/2	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
23	Orthographic Projections	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.

Page	Slide title	How to read it
24	Axometric Projections	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
25	Isometric Projections	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
26	Oblique Projection 1/2	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
27	Oblique Projection 2/2	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
28	Properties of Parallel Projections	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
29	Perspective Projections 1/3	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
30	Perspective Projections 2/3	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
31	Perspective Projections 3/3	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
32	4.3 Camera Modeling	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
33	Camera Modeling	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
34	View Volume	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
35	Projection Specification	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
36	View Angle	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
37	Clipping Planes	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
38	4.4 Specifying Projections in OpenGL	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.

Page	Slide title	How to read it
39	Projection Matrix Generation 1/3	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
40	Projection Matrix Generation 2/3	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
41	Projection Matrix Generation 3/3	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
42	Orthographic Matrix Specification	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
43	4.5 Orthographic Projection Derivation	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
44	Canonical View Volume	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
45	Orthographic Projection 1/4	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
46	Orthographic Projection 2/4	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
47	Orthographic Projection 3/4	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
48	Orthographic Projection 4/4	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
49	4.6 Perspective Projection Derivation	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
50	Perspective Projection	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
51	Transformation of z-Values 1/2	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
52	Transformation of z-Values 2/2	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
53	Transformation of x- and y-Values 1/2	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.

Page	Slide title	How to read it
54	Transformation of x- and y-Values 2/2	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
55	4.7 Viewport Transformation	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
56	Screen Coordinate Mapping	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
57	Viewport Transformation 1/2	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
58	Viewport Transformation 2/2	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
59	volume, we can differentiate different projection types	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
60	Literature and other sources used in this chapter	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
61	- Text Books	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?

04.1 Linear Perspective and Planar Projections

Source location: Sections 4.1-4.2

Commentary

Perspective projection models the visual effect that farther objects appear smaller. Orthographic projection removes this distance-based size change and keeps parallel lines parallel. Both are useful, but they communicate different spatial relationships.

Planar projection means mapping points onto an image plane. The lecture's art examples are not decoration; they show that projection is a geometric rule for constructing an image from a viewpoint.

Mental model

Projection is the rule that turns 3D positions into image-plane positions.

Check yourself

What visual cue does perspective projection add that orthographic projection lacks?

04.2 Camera Modeling

Source location: Section 4.3

Commentary

A virtual camera is defined by position, orientation, projection type, viewing volume, and image/viewport settings. The view transform places the world relative to the camera. The projection transform maps that view into clip coordinates.

Near and far clipping planes are part of the camera model. They decide what range of depths is represented and strongly affect depth buffer precision. A careless near/far setup can create z-fighting even if the scene geometry is correct.

Mental model

A camera is not only an eye position; it is also a volume and a mapping rule.

Check yourself

Why can changing the near plane affect depth artifacts?

04.3 Specifying Projections in OpenGL

Source location: Section 4.4

Commentary

In OpenGL, projection is usually encoded in a projection matrix sent to a shader. The matrix maps view-space coordinates to clip space. Clipping and the perspective divide then lead toward normalized device coordinates.

Aspect ratio and field of view must match the intended viewport. If the aspect ratio is wrong, objects appear stretched. If the field of view is extreme, the image can look distorted even though the math is functioning.

Mental model

The projection matrix is the camera lens encoded as linear algebra in homogeneous coordinates.

Check yourself

What symptom suggests that the projection aspect ratio does not match the window?

04.4 Orthographic and Perspective Derivations

Source location: Sections 4.5-4.6

Commentary

The derivations are there to explain where the matrix entries come from. Orthographic projection maps a box-shaped view volume into normalized coordinates. Perspective projection maps a frustum and uses the homogeneous component for the later divide.

You do not need to treat these matrices as random formulas. Each part maps a coordinate range into a standard range. The perspective matrix additionally arranges w so that the perspective divide creates foreshortening.

Mental model

Projection matrices normalize a viewing volume; perspective also prepares the divide by w .

Check yourself

What does the perspective divide do to x and y coordinates?

04.5 Viewport Transformation

Source location: Section 4.7

Commentary

After normalized device coordinates, the viewport transform maps the normalized range to actual window coordinates. This is the last geometric mapping before rasterization uses the target grid.

Separating projection from viewport mapping helps debugging. A wrong projection can produce wrong normalized coordinates; a wrong viewport can map otherwise valid coordinates into the wrong part of the window.

Mental model

Projection decides normalized position; viewport decides where that position lands on the screen.

Check yourself

Why is resizing a window related to both viewport and projection settings?

End-of-lecture summary

If you remember only one thing from Lecture 04, remember this: This lecture explains how a 3D view becomes a 2D image. Projection is where camera geometry, homogeneous coordinates, clipping planes, depth precision, and viewport mapping meet.

Before moving on, you should be able to explain the lecture without reading slide bullets aloud. Use the vocabulary from the slides, but add causal links: why a step exists, what data it consumes, what it produces, and what can go wrong.

Lecture 05 - Clipping

This lecture focuses on algorithms that remove or trim geometry against boundaries. Clipping is a geometric operation before rasterization, not a visibility test between overlapping objects.

Source chunk	course_text_parts/03_lectures/05_clipping.txt
Extracted slide pages	56
Reader purpose	Commentary and causal explanation for human reading

How to read this lecture

- Read the section commentary before looking at the raw slide bullets.
- Use the mental model as the short version you should remember.
- Answer the check question without notes before continuing.

Slide walkthrough

Each row gives a short reading comment for one extracted slide page. Use it to turn sparse slide bullets into a human-readable path through the lecture.

Page	Slide title	How to read it
1	Untitled slide	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
2	Clipping Projected Geometry 1/2	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
3	Clipping Projected Geometry 2/2	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
4	5.1 Cohen-Sutherland Line Clipping	Read this as boundary logic. Identify what is inside, what is outside, and when new intersection vertices are created.
5	5.1 Cohen-Sutherland Line Clipping	Read this as boundary logic. Identify what is inside, what is outside, and when new intersection vertices are created.
6	Brute-Force Line Clipping 1/2	Read this as boundary logic. Identify what is inside, what is outside, and when new intersection vertices are created.
7	Brute-Force Line Clipping 2/2	Read this as boundary logic. Identify what is inside, what is outside, and when new intersection vertices are created.
8	Endpoint Analysis 1/2	Read this as boundary logic. Identify what is inside, what is outside, and when new intersection vertices are created.

Page	Slide title	How to read it
9	Endpoint Analysis 2/2	Read this as boundary logic. Identify what is inside, what is outside, and when new intersection vertices are created.
10	Binary Coding of Half-Planes 1/3	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
11	Binary Coding of Half-Planes 2/3	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
12	Binary Coding of Half-Planes 3/3	Read this as boundary logic. Identify what is inside, what is outside, and when new intersection vertices are created.
13	Pseudo Code	Read this as boundary logic. Identify what is inside, what is outside, and when new intersection vertices are created.
14	Example	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
15	Evaluation	Read this as boundary logic. Identify what is inside, what is outside, and when new intersection vertices are created.
16	5.2 Cyrus-Beck Line Clipping	Read this as boundary logic. Identify what is inside, what is outside, and when new intersection vertices are created.
17	Cyrus-Beck Line Clipping	Read this as boundary logic. Identify what is inside, what is outside, and when new intersection vertices are created.
18	Parametric Line Representation	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
19	Intersection Point Calculation 1/2	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
20	Intersection Point Calculation 2/2	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
21	Determine Relevant t-Values	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
22	Pseudo Code	Read this as boundary logic. Identify what is inside, what is outside, and when new intersection vertices are created.
23	Axis-Parallel Clipping Region	Read this as boundary logic. Identify what is inside, what is outside, and when new intersection vertices are created.

Page	Slide title	How to read it
24	Comparison with Cohen-Sutherland	Read this as boundary logic. Identify what is inside, what is outside, and when new intersection vertices are created.
25	5.3 Sutherland-Hodgman Polygon Clipping	Read this as boundary logic. Identify what is inside, what is outside, and when new intersection vertices are created.
26	Sutherland-Hodgman Polygon Clipping 1/2	Read this as boundary logic. Identify what is inside, what is outside, and when new intersection vertices are created.
27	Sutherland-Hodgman Polygon Clipping 2/2	Read this as boundary logic. Identify what is inside, what is outside, and when new intersection vertices are created.
28	Four Cases	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
29	Proceeding	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
30	Pipeline Version	Read this as boundary logic. Identify what is inside, what is outside, and when new intersection vertices are created.
31	Pseudo Code	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
32	Inside Test	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
33	Intersection Computation	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
34	5.4 Weiler-Atherton Polygon Clipping	Read this as boundary logic. Identify what is inside, what is outside, and when new intersection vertices are created.
35	Weiler-Atherton Polygon Clipping	Read this as boundary logic. Identify what is inside, what is outside, and when new intersection vertices are created.
36	Example Run	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
37	Intersection Types	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
38	Contour Processing	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?

Page	Slide title	How to read it
39	Evaluation	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
40	5.5 Greiner-Hormann Polygon Clipping	Read this as boundary logic. Identify what is inside, what is outside, and when new intersection vertices are created.
41	Greiner-Hormann	Read this as boundary logic. Identify what is inside, what is outside, and when new intersection vertices are created.
42	Chalk Wagon Metaphor	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
43	Contour Detection	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
44	Set Operations	Read this as boundary logic. Identify what is inside, what is outside, and when new intersection vertices are created.
45	Data Structure 1/2	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
46	Data Structure 2/2	Read this as boundary logic. Identify what is inside, what is outside, and when new intersection vertices are created.
47	Proceeding	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
48	Pseudo Code	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
49	Winding Number 1/3	Read this as an occlusion decision. Decide whether the method works in object space, image space, or per-fragment depth space.
50	Winding Number 2/3	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
51	Winding Number 3/3	Read this as boundary logic. Identify what is inside, what is outside, and when new intersection vertices are created.
52	Special Cases	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
53	Evaluation	Read this as boundary logic. Identify what is inside, what is outside, and when new intersection vertices are created.

Page	Slide title	How to read it
54	by exploiting binary outcodes	Read this as boundary logic. Identify what is inside, what is outside, and when new intersection vertices are created.
55	Literature and other sources used in this chapter	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
56	Addison-Wesley 2013.	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?

05.1 Cohen-Sutherland Line Clipping

Source location: Section 5.1

Commentary

Cohen-Sutherland assigns outcodes to line endpoints based on which side of the clipping window they lie on. These codes allow quick accept, quick reject, or partial clipping. The algorithm is efficient because many cases are decided before computing intersections.

The exam-friendly idea is the logic of region codes: if both endpoints are inside, keep the segment. If the bitwise AND of the outcodes is nonzero, both endpoints share an outside region and the segment can be rejected. Otherwise, compute an intersection and continue.

Mental model

Outcodes are a cheap classification before doing intersection math.

Check yourself

What does a nonzero bitwise AND of two endpoint outcodes imply?

05.2 Cyrus-Beck Line Clipping

Source location: Section 5.2

Commentary

Cyrus-Beck treats the line parametrically and clips against convex boundaries. Instead of repeatedly using region codes, it finds entering and leaving parameter values along the line.

The key is to reason about the line parameter t . A clipped line segment is not a new unrelated line; it is a restricted interval of the original parametric line. Boundary tests shrink that interval.

Mental model

Clipping a parametric line means narrowing the valid t interval.

Check yourself

Why does Cyrus-Beck naturally require convex clipping regions?

05.3 Sutherland-Hodgman Polygon Clipping

Source location: Section 5.3

Commentary

Sutherland-Hodgman clips a polygon against one boundary at a time. For each edge of the polygon, it decides whether vertices are inside or outside and emits zero, one, or two vertices depending on transitions across the boundary.

This algorithm is easiest to understand as a stream processor. Feed in a polygon, clip it against the left boundary, feed the result into the right boundary, and so on. Each boundary can add intersection vertices.

Mental model

Polygon clipping is repeated edge-by-edge filtering plus intersection insertion.

Check yourself

When can polygon clipping increase the number of vertices?

05.4 Weiler-Atherton and Greiner-Hormann

Source location: Sections 5.4-5.5

Commentary

These algorithms address more complex polygon clipping scenarios, especially when polygon relationships are not as simple as convex-window clipping. They organize intersections and traversal rules to produce correct output boundaries.

For preparation, focus on why simpler algorithms are not enough. Complex polygon intersection can require following alternating boundaries of subject and clipping polygons. That is a topology problem, not only a line-intersection problem.

Mental model

Complex polygon clipping is about traversing boundary networks after intersections are known.

Check yourself

Why are intersection ordering and traversal rules important for polygon clipping?

End-of-lecture summary

If you remember only one thing from Lecture 05, remember this: This lecture focuses on algorithms that remove or trim geometry against boundaries. Clipping is a geometric operation before rasterization, not a visibility test between overlapping objects.

Before moving on, you should be able to explain the lecture without reading slide bullets aloud. Use the vocabulary from the slides, but add causal links: why a step exists, what data it consumes, what it produces, and what can go wrong.

Lecture 06 - Rasterization

This lecture explains how continuous primitives become discrete fragments on a grid. Rasterization is the bridge from geometric descriptions to sample-based image generation.

Source chunk	course_text_parts/03_lectures/06_rasterization.txt
Extracted slide pages	61
Reader purpose	Commentary and causal explanation for human reading

How to read this lecture

- Read the section commentary before looking at the raw slide bullets.
- Use the mental model as the short version you should remember.
- Answer the check question without notes before continuing.

Slide walkthrough

Each row gives a short reading comment for one extracted slide page. Use it to turn sparse slide bullets into a human-readable path through the lecture.

Page	Slide title	How to read it
1	Untitled slide	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
2	Rasterization 1/2	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
3	Rasterization 2/2	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
4	6.1 Line Rasterization	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
5	6.1 Line Rasterization	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
6	Line Rasterization	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
7	Quality Criteria	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
8	Naïve Algorithm 1/2	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.

Page	Slide title	How to read it
9	Naïve Algorithm 2/2	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
10	Digital Difference Analyzer (DDA) 1/3	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
11	Digital Difference Analyzer (DDA) 2/3	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
12	Digital Difference Analyzer (DDA) 3/3	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
13	Midpoint-Line Algorithm 1/2	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
14	Midpoint-Line Algorithm 2/2	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
15	Implicit Function	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
16	Decision Variable ■	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
17	Update of ■ for ■	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
18	Update of ■ for ■■	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
19	Proceeding	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
20	Initializing ■	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
21	Using ■	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
22	Pseudo Code – Simple Version	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
23	Example	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.

Page	Slide title	How to read it
24	Evaluation	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
25	Pseudo Code - Reversible	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
26	6.2 Triangle Edge Rasterization	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
27	Triangle Edges 1/2	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
28	Triangle Edges 2/2	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
29	Edge Rasterization 1/3	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
30	Edge Rasterization 2/3	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
31	Edge Rasterization 3/3	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
32	6.3 Region Filling	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
33	Region Types 1/2	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
34	Region Types 2/2	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
35	Region Spezifikation	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
36	Flood Fill Algorithm	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
37	Boundary Fill Algorithm	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
38	Fill Algorithms	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?

Page	Slide title	How to read it
39	6.4 Scanline-Based Triangle Rasterization	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
40	Triangle Rasterization	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
41	Scanline-Based Rasterization	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
42	Algorithm Overview	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
43	Slope Calculation	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
44	Pseudo Code	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
45	Segment Rasterization	Read this as the main data-flow story: scene/model data is processed step by step until only valid framebuffer updates remain.
46	Vertex Attribute Interpolation	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
47	Incremental Interpolation 1/3	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
48	Incremental Interpolation 2/3	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
49	Incremental Interpolation 3/3	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
50	6.5 Tile-Based Triangle Rasterization	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
51	Tile-Based Triangle Rasterization	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
52	Edge-Based Triangle Representation 1/2	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
53	Edge-Based Triangle Representation 2/2	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.

Page	Slide title	How to read it
54	Rasterization Procedure	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
55	between adjacent triangles	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
56	Literature and other sources used in this chapter	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
57	Practice, Addison-Wesley.	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
58	Slide for Future: Sustainability in CG (@EG25)	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
59	Slide for Future: Sustainability in CG (@EG25)	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
60	Slide for Future: Sustainability in CG (@EG25)	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
61	Slide for Future: Sustainability in CG (@EG25)	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?

06.1 Line Rasterization

Source location: Section 6.1

Commentary

A mathematical line has infinitely many points, but a raster display has a finite grid. Line rasterization decides which grid cells or samples best approximate the ideal line. The algorithm must balance accuracy, speed, and consistency.

Incremental algorithms avoid recomputing expensive formulas for every pixel. The important idea is to step along one axis and update an error term that decides when to step along the other axis.

Mental model

Rasterizing a line means approximating a continuous path by grid decisions.

Check yourself

Why are incremental error updates useful in line rasterization?

06.2 Triangle Edge Rasterization

Source location: Section 6.2

Commentary

Triangles are the core primitive in real-time rendering. Edge functions or half-space tests decide whether a sample lies inside the triangle. This allows the rasterizer to test coverage systematically.

Coverage is not final visibility. A covered sample becomes a fragment candidate. Depth testing, stencil testing, blending, and other operations still decide whether the framebuffer changes.

Mental model

Triangle rasterization answers the question: which samples are inside this projected triangle?

Check yourself

What later stage can reject a fragment after triangle coverage succeeds?

06.3 Region Filling

Source location: Section 6.3

Commentary

Region filling generalizes the idea of determining interior samples. The algorithm must identify connected areas or spans that should receive color. This connects rasterization to scan conversion and image-space reasoning.

The practical issue is avoiding gaps, double fills, or inconsistent boundary handling. Small choices at edges can become visible artifacts when many primitives meet.

Mental model

Filling is about turning boundary descriptions into consistent interior samples.

Check yourself

Why can inconsistent edge rules create cracks between adjacent primitives?

06.4 Scanline-Based Triangle Rasterization

Source location: Section 6.4

Commentary

Scanline rasterization processes horizontal rows of the image. For each row that crosses a triangle, the algorithm finds the covered interval and fills the span. Attribute interpolation can be updated along edges and across each span.

The method is intuitive because it follows the memory layout of many images. It also demonstrates why interpolation is tied to rasterization: once you know a sample location inside the primitive, you can interpolate depth, color, normals, or texture coordinates.

Mental model

Each scanline asks: where does this triangle enter and leave this row?

Check yourself

Which attributes might be interpolated while filling triangle spans?

06.5 Tile-Based Triangle Rasterization

Source location: Section 6.5

Commentary

Tile- or block-based approaches group pixels into small regions. This can improve locality and parallel work distribution. Modern GPUs often reason in blocks or tiles internally because rendering is massively parallel.

For the exam, connect this to efficiency. The mathematical goal is still coverage, but the implementation is organized to use hardware resources well.

Mental model

Block-based rasterization keeps the same coverage problem but changes the work organization.

Check yourself

Why is block organization attractive for parallel graphics hardware?

End-of-lecture summary

If you remember only one thing from Lecture 06, remember this: This lecture explains how continuous primitives become discrete fragments on a grid. Rasterization is the bridge from geometric descriptions to sample-based image generation.

Before moving on, you should be able to explain the lecture without reading slide bullets aloud. Use the vocabulary from the slides, but add causal links: why a step exists, what data it consumes, what it produces, and what can go wrong.

Lecture 07 - Visibility Determination

This lecture asks which surfaces are visible from a viewpoint. Rasterization can generate many fragment candidates, but visibility decides which ones should affect the image.

Source chunk	course_text_parts/03_lectures/07_visibility-determination.txt
Extracted slide pages	69
Reader purpose	Commentary and causal explanation for human reading

How to read this lecture

- Read the section commentary before looking at the raw slide bullets.
- Use the mental model as the short version you should remember.
- Answer the check question without notes before continuing.

Slide walkthrough

Each row gives a short reading comment for one extracted slide page. Use it to turn sparse slide bullets into a human-readable path through the lecture.

Page	Slide title	How to read it
1	Untitled slide	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
2	Visibility Determination	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
3	Object-Based Algorithms	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
4	Image-Based Algorithms	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
5	7.1 Object-Based Algorithms	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
6	7.1 Object-Based Algorithms	Read this as boundary logic. Identify what is inside, what is outside, and when new intersection vertices are created.
7	Painter's Algorithm 1/3	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
8	Painter's Algorithm 2/3	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?

Page	Slide title	How to read it
9	Painter's Algorithm 3/3	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
10	Weiler-Atherton Algorithm 1/3	Read this as boundary logic. Identify what is inside, what is outside, and when new intersection vertices are created.
11	Weiler-Atherton Algorithm 2/3	Read this as boundary logic. Identify what is inside, what is outside, and when new intersection vertices are created.
12	Weiler-Atherton Algorithm 3/3	Read this as boundary logic. Identify what is inside, what is outside, and when new intersection vertices are created.
13	Back Face Culling 1/2	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
14	Back Face Culling 2/2	Read this as the main data-flow story: scene/model data is processed step by step until only valid framebuffer updates remain.
15	Robert's Algorithm	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
16	Assessment Robert's Algorithm	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
17	7.2 Binary Space Partitioning	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
18	Spatial Data Structures	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
19	BSP Trees 1/2	Read this as an occlusion decision. Decide whether the method works in object space, image space, or per-fragment depth space.
20	BSP Trees 2/2	Read this as an occlusion decision. Decide whether the method works in object space, image space, or per-fragment depth space.
21	Node Data Structure	Read this as an occlusion decision. Decide whether the method works in object space, image space, or per-fragment depth space.
22	BSP Tree Construction	Read this as an occlusion decision. Decide whether the method works in object space, image space, or per-fragment depth space.
23	BSP Tree Construction - Example	Read this as an occlusion decision. Decide whether the method works in object space, image space, or per-fragment depth space.

Page	Slide title	How to read it
24	BSP Tree Construction – Pseudo Code	Read this as an occlusion decision. Decide whether the method works in object space, image space, or per-fragment depth space.
25	Visibility Determination	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
26	Visibility Determination – Pseudo Code	Read this as an occlusion decision. Decide whether the method works in object space, image space, or per-fragment depth space.
27	BSP Tree Traversal - Example	Read this as an occlusion decision. Decide whether the method works in object space, image space, or per-fragment depth space.
28	Splitting Planes Selection	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
29	Traversal Optimization	Read this as an occlusion decision. Decide whether the method works in object space, image space, or per-fragment depth space.
30	Assessment BSP Trees	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
31	7.3 Warnock Algorithm	Read this as an occlusion decision. Decide whether the method works in object space, image space, or per-fragment depth space.
32	Warnock Algorithm	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
33	Region-Based Visibility Determination	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
34	Procedure	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
35	Pseudo Code	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
36	Assessment Warnock Algorithm	Read this as an occlusion decision. Decide whether the method works in object space, image space, or per-fragment depth space.
37	7.4 Depth Buffer Algorithm	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
38	Depth Buffer Algorithm 1/3	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.

Page	Slide title	How to read it
39	Depth Buffer Algorithm 2/3	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
40	Depth Buffer Algorithm 3/3	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
41	Pseudo Code – Depth Buffer Test	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
42	OpenGL Depth Buffer	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
43	Depth Buffer Precision	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
44	Z-Fighting 1/2	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
45	Z-Fighting 2/2	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
46	Assessment Depth Buffering	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
47	7.5 Depth Buffer Extensions	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
48	Semi-Transparent Objects 1/2	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
49	Semi-Transparent Objects 2/2	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
50	Order-Independent Transparency 1/4	Read this as an occlusion decision. Decide whether the method works in object space, image space, or per-fragment depth space.
51	Order-Independent Transparency 2/4	Read this as an occlusion decision. Decide whether the method works in object space, image space, or per-fragment depth space.
52	Order-Independent Transparency 3/4	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
53	Order-Independent Transparency 4/4	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?

Page	Slide title	How to read it
54	Generating Haloes 1/2	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
55	Generating Haloes 2/2	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
56	7.6 Ray Casting	Read this as an occlusion decision. Decide whether the method works in object space, image space, or per-fragment depth space.
57	Ray Casting Principle	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
58	Ray Initialization 1/3	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
59	Ray Initialization 2/3	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
60	Ray Initialization 3/3	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
61	Intersection Calculation	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
62	Implicit Surface Intersection Calculation	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
63	Intersection Calculation - Triangle	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
64	Pseudo Code – Analytic Intersection Computation	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
65	Pseudo Code - Iterative Intersection Calculation (Ray Marching)	Read this as an occlusion decision. Decide whether the method works in object space, image space, or per-fragment depth space.
66	Assessment Ray Casting	Read this as an occlusion decision. Decide whether the method works in object space, image space, or per-fragment depth space.
67	but are crucial for accurate visibility determination	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
68	Literature and other sources used in this chapter	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?

Page	Slide title	How to read it
69	Practice, Addison-Wesley.	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?

07.1 Object-Based Algorithms

Source location: Section 7.1

Commentary

Object-based visibility algorithms reason about geometry before or while comparing surfaces. They can sort, split, or reject objects based on spatial relations. This is different from simply letting every fragment fight in the depth buffer.

The value of object-based reasoning is reducing work or establishing correct order. The difficulty is that geometry can overlap in complicated ways, so simple global ordering is not always possible.

Mental model

Object-based visibility tries to solve visibility with geometry before final pixels are written.

Check yourself

Why can intersecting polygons make simple depth sorting fail?

07.2 Binary Space Partitioning

Source location: Section 7.2

Commentary

A BSP tree recursively divides space with planes. Once built, it can help traverse geometry in a view-dependent order. This is useful for visibility and ordering because the tree encodes spatial relationships.

The cost is preprocessing and possible splitting of geometry. BSP is a good example of trading memory and setup work for faster or more structured visibility decisions later.

Mental model

A BSP tree stores a recursive answer to 'which side of this plane is the geometry on?'

Check yourself

Why can building a BSP tree require splitting polygons?

07.3 Warnock Algorithm

Source location: Section 7.3

Commentary

Warnock's algorithm works in image space by subdividing regions until visibility is simple enough to decide. If a region is ambiguous, split it. If it is simple, fill it.

This illustrates a general graphics strategy: recursively reduce a hard global problem into smaller local problems. The algorithm is less central in modern OpenGL practice than the depth buffer, but it helps contrast image-space and object-space approaches.

Mental model

When a region is too complicated, subdivide until the answer becomes simple.

Check yourself

What makes Warnock's algorithm image-space rather than object-space?

07.4 Depth Buffer Algorithm

Source location: Section 7.4

Commentary

The depth buffer stores the closest accepted depth at each sample or pixel. For each fragment, compare its depth to the stored depth. If it passes, update the color and depth; otherwise discard it.

The strength of the depth buffer is simplicity and hardware efficiency. The weakness is precision: depth values are finite, and projection distributes precision unevenly. Bad near/far settings can create z-fighting.

Mental model

The depth buffer is a per-sample competition for closest visible fragment.

Check yourself

Why should the near plane not be unnecessarily close to the camera?

07.5 Depth Extensions and Ray Casting

Source location: Sections 7.5-7.6

Commentary

Depth buffer extensions refine or adapt depth-based visibility, for example by handling precision, transparency, or multiple layers more carefully. Standard depth testing alone is not a complete solution for every visibility situation.

Ray casting takes the opposite direction from rasterization: for each image sample, cast a ray into the scene and find the closest intersection. This naturally solves primary visibility but requires intersection work instead of raster coverage work.

Mental model

Rasterization pushes primitives to pixels; ray casting pulls visibility from pixels into the scene.

Check yourself

Why is transparency harder than opaque nearest-surface visibility?

End-of-lecture summary

If you remember only one thing from Lecture 07, remember this: This lecture asks which surfaces are visible from a viewpoint. Rasterization can generate many fragment candidates, but visibility decides which ones should affect the image.

Before moving on, you should be able to explain the lecture without reading slide bullets aloud. Use the vocabulary from the slides, but add causal links: why a step exists, what data it consumes, what it produces, and what can go wrong.

Lecture 08 - Local Illumination

This lecture explains how visible surface points receive color from local light, material, normal, and view information. It is about shading a point, not solving all global light transport.

Source chunk	course_text_parts/03_lectures/08_local-illumination.txt
Extracted slide pages	62
Reader purpose	Commentary and causal explanation for human reading

How to read this lecture

- Read the section commentary before looking at the raw slide bullets.
- Use the mental model as the short version you should remember.
- Answer the check question without notes before continuing.

Slide walkthrough

Each row gives a short reading comment for one extracted slide page. Use it to turn sparse slide bullets into a human-readable path through the lecture.

Page	Slide title	How to read it
1	Untitled slide	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
2	Local Illumination	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
3	8.1 Physics of Light	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
4	8.1 Physics of Light	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
5	Electromagnetic Radiation	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
6	Visible Light	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
7	Visible Light	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
8	Light Spectra	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.

Page	Slide title	How to read it
9	Black-Body Radiation	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
10	Black-Body Radiation	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
11	Spectral Density Function (SDF)	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
12	Light Perception	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
13	Interpreting SDFs	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
14	Representing SDFs	Read this as an occlusion decision. Decide whether the method works in object space, image space, or per-fragment depth space.
15	8.2 Light Sources	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
16	Light Sources	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
17	Point Light Sources	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
18	Cone Light Sources	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
19	Light Attenuation	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
20	Area Light Sources	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
21	Directed Light	Read this as an occlusion decision. Decide whether the method works in object space, image space, or per-fragment depth space.
22	Local vs. Global Illumination	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
23	Ambient Light	Read this as an occlusion decision. Decide whether the method works in object space, image space, or per-fragment depth space.

Page	Slide title	How to read it
24	8.3 Material Models	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
25	Material Models	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
26	Materials vs. Textures	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
27	Light Material Interaction 1/3	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
28	Light Material Interaction 2/3	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
29	Light Material Interaction 3/3	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
30	Reflection = SDF x SRF	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
31	Reflection Modeling 1/2	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
32	Reflection Modeling 2/2	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
33	8.4 Phong Illumination Model	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
34	Empirical Observations	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
35	Illumination Models	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
36	yaR	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
37	Diffuse Intensity 2/3	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
38	Diffuse Intensity 3/3	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?

Page	Slide title	How to read it
39	Ambient Intensity 1/2	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
40	Ambient Intensity 2/2	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
41	Specular Intensity 1/3	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
42	Reflection Vector ■■	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
43	Specular Intensity 2/3	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
44	Specular Exponent ■	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
45	Specular Intensity 3/3	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
46	Blinn-Phong Illumination Model 1/2	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
47	Blinn-Phong Illumination Model 2/2	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
48	Phong Illumination Model	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
49	Phong Illumination Model	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
50	8.5 Shading	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
51	Shading	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
52	Shading Computation	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
53	Flat Shading	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.

Page	Slide title	How to read it
54	Gouraud Shading 1/5	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
55	Gouraud Shading 2/5	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
56	Gouraud Shading 3/5	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
57	Gouraud Shading 4/5	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
58	Gouraud Shading 5/5	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
59	Phong Shading 1/2	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
60	Phong Shading 2/2	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
61	Literature and other sources used in this chapter	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
62	Lesen und Ausprobieren	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.

08.1 Physics of Light

Source location: Section 8.1

Commentary

The physics slides provide intuition for light as energy interacting with surfaces. For this course, the goal is not full physical simulation but a usable model for real-time rendering.

Important terms are reflection, absorption, wavelength/color, intensity, and direction. These terms become shader inputs or material parameters later. The simplified model must still preserve the relation between light direction, surface orientation, and observed brightness.

Mental model

Shading is an approximation of light-surface interaction that is cheap enough for rendering.

Check yourself

Why does surface orientation affect diffuse brightness?

08.2 Light Sources

Source location: Section 8.2

Commentary

Light source models define where light comes from and how its direction and intensity are computed. Directional lights approximate distant sources with parallel rays. Point lights emit from a position and often include attenuation. Spotlights add a directional cone.

In shader code, the type of light determines how you compute the light vector. A directional light can use a fixed direction; a point light requires subtracting the surface position from the light position.

Mental model

Different light types mainly change how the light direction and intensity are computed at the surface point.

Check yourself

What is the difference between a directional light vector and a point light vector?

08.3 Material Models

Source location: Section 8.3

Commentary

A material model tells the shader how a surface responds to light. Diffuse material scatters light broadly; specular material creates view-dependent highlights; ambient terms approximate background illumination.

Material coefficients are not arbitrary decoration. They scale the contribution of each lighting term. If the specular coefficient is zero, the surface should not show a specular highlight under that model.

Mental model

Light asks what arrives; material answers how the surface reacts.

Check yourself

Which material parameter would you adjust to reduce shiny highlights?

08.4 Phong Illumination Model

Source location: Section 8.4

Commentary

The Phong illumination model combines ambient, diffuse, and specular terms. Diffuse lighting uses the angle between normal and light direction. Specular lighting also depends on the viewer because highlights move with the view direction.

The formula is less important than the decomposition. Ambient is a rough constant approximation. Diffuse is orientation-dependent matte reflection. Specular is view-dependent shininess. A correct answer should name the vectors and normalize them.

Mental model

Phong-style lighting is a sum of simple terms, each modeling a different visual effect.

Check yourself

Why does the specular term depend on the view direction?

08.5 Shading

Source location: Section 8.5

Commentary

Shading decides where the illumination calculation is evaluated and how results are interpolated. Flat shading uses one value per primitive. Gouraud shading evaluates at vertices and interpolates colors. Phong shading interpolates normals and evaluates lighting per fragment.

Per-fragment shading is more expensive but can represent highlights more accurately. This connects directly to the pipeline: interpolation during rasterization provides the data used by the fragment shader.

Mental model

The shading method decides what is computed at vertices and what is computed at fragments.

Check yourself

Why can Gouraud shading miss a small specular highlight?

End-of-lecture summary

If you remember only one thing from Lecture 08, remember this: This lecture explains how visible surface points receive color from local light, material, normal, and view information. It is about shading a point, not solving all global light transport.

Before moving on, you should be able to explain the lecture without reading slide bullets aloud. Use the vocabulary from the slides, but add causal links: why a step exists, what data it consumes, what it produces, and what can go wrong.

Lecture 09 - Texturing

This lecture treats textures as sampled data for shading, not merely images glued onto objects. It connects UV coordinates, sampling, filtering, mipmaps, OpenGL texture state, and advanced texture uses.

Source chunk	course_text_parts/03_lectures/09_texturing.txt
Extracted slide pages	133
Reader purpose	Commentary and causal explanation for human reading

How to read this lecture

- Read the section commentary before looking at the raw slide bullets.
- Use the mental model as the short version you should remember.
- Answer the check question without notes before continuing.

Slide walkthrough

Each row gives a short reading comment for one extracted slide page. Use it to turn sparse slide bullets into a human-readable path through the lecture.

Page	Slide title	How to read it
1	Untitled slide	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
2	Textures: Sampled Data for Shading	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
3	Texturing Example	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
4	Texture Mapping	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
5	Why Textures Matter	Read this as sampled data. Identify the texture coordinates, sampling rule, filtering mode, and shader interpretation.
6	Textures Are More Than Color Images	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
7	Texturing Demo	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
8	Overview	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.

Page	Slide title	How to read it
9	9.1 Texture Objects, Texels, Formats, and Color Space	Read this as sampled data. Identify the texture coordinates, sampling rule, filtering mode, and shader interpretation.
10	From Image File to GPU Texture Object	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
11	Texels, Channels, and Meaning	Read this as sampled data. Identify the texture coordinates, sampling rule, filtering mode, and shader interpretation.
12	Texture Targets and Dimensionality	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
13	Internal Texture Formats	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
14	Numeric Texture Representations	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
15	Color Textures vs. Data Textures	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
16	2D Texture Access	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
17	Bilinear Interpolation	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
18	Untitled slide	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
19	Texture Memory Footprint	Read this as sampled data. Identify the texture coordinates, sampling rule, filtering mode, and shader interpretation.
20	Texture Compression and Asset Pipelines	Read this as sampled data. Identify the texture coordinates, sampling rule, filtering mode, and shader interpretation.
21	Steps Involved in Texturing	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
22	9.2 Texture Coordinates, UV Mapping, and Wrapping	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
23	Texture Coordinate System 1/2	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.

Page	Slide title	How to read it
24	Texture Coordinate System 2/2	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
25	Texture Coordinate Specification	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
26	Disc Parametrization	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
27	Cylinder Parametrization	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
28	Two-Part Mapping 1/2	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
29	Two-Part Mapping 2/2	Read this as sampled data. Identify the texture coordinates, sampling rule, filtering mode, and shader interpretation.
30	Two-Part Mapping Example	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
31	Projection Calculation Variations	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
32	Texture Coordinate Transformation	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
33	Demo - Texture Coordinate Transformation	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
34	Height Field Texturing Example	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
35	9.3 Sampling, Filtering, Mipmaps, and Anisotropy	Read this as sampled data. Identify the texture coordinates, sampling rule, filtering mode, and shader interpretation.
36	Forward vs. Inverse Mapping	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
37	Forward Mapping	Read this as sampled data. Identify the texture coordinates, sampling rule, filtering mode, and shader interpretation.
38	Inverse Mapping 1/2	Read this as sampled data. Identify the texture coordinates, sampling rule, filtering mode, and shader interpretation.

Page	Slide title	How to read it
39	Inverse Mapping 2/2	Read this as sampled data. Identify the texture coordinates, sampling rule, filtering mode, and shader interpretation.
40	Magnification vs. Minification	Read this as sampled data. Identify the texture coordinates, sampling rule, filtering mode, and shader interpretation.
41	Nearest vs. Linear Interpolation	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
42	Minification Filtering Requirements	Read this as sampled data. Identify the texture coordinates, sampling rule, filtering mode, and shader interpretation.
43	Demo - Magnification Filtering: Nearest vs Linear	Read this as sampled data. Identify the texture coordinates, sampling rule, filtering mode, and shader interpretation.
44	MIP Mapping (multum in parvo = 'much in little')	Read this as sampled data. Identify the texture coordinates, sampling rule, filtering mode, and shader interpretation.
45	MIP Map Pyramid 1/2	Read this as sampled data. Identify the texture coordinates, sampling rule, filtering mode, and shader interpretation.
46	MIP Map Pyramid 2/2	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
47	MIP Mapping Memory	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
48	MIP Mapping Procedure	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
49	LoD Calculation 1/2	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
50	LoD Calculation 2/2	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
51	Demo - Minification, Mipmaps, and Mipmap Level Visualization	Read this as sampled data. Identify the texture coordinates, sampling rule, filtering mode, and shader interpretation.
52	Anisotropic Filtering	Read this as sampled data. Identify the texture coordinates, sampling rule, filtering mode, and shader interpretation.
53	Demo - Anisotropic Filtering	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.

Page	Slide title	How to read it
54	9.4 Modern OpenGL Texture Pipeline	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
55	OpenGL Texturing	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
56	OpenGL Minification & Magnification 1/3	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
57	OpenGL Minification & Magnification 2/3	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
58	OpenGL Minification & Magnification 3/3	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
59	Demo - Texture Units and Sampler Uniforms	Read this as sampled data. Identify the texture coordinates, sampling rule, filtering mode, and shader interpretation.
60	Texture Wrapping 1/4	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
61	Texture Wrapping 2/4	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
62	Texture Wrapping 3/4	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
63	Texture Wrapping 4/4	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
64	Texel Value Usage	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
65	Texturing in GLSL Shaders 1/3	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
66	Texturing in GLSL Shaders 2/3	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
67	Texturing in GLSL Shaders 3/3	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
68	9.5 Normal Mapping	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.

Page	Slide title	How to read it
69	Motivation	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
70	Mesh Compression	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
71	Normal Mapping 1/2	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
72	Normal Mapping 2/2	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
73	Normal Maps	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
74	Tangent Space	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
75	TBN Matrix	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
76	Untitled slide	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
77	Tangent Vector Derivation 1/2	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
78	Untitled slide	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
79	Tangent Vector Derivation 2/2	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
80	Untitled slide	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
81	Observations	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
82	Variations	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
83	9.6 Environment Mapping	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.

Page	Slide title	How to read it
84	Environment Mapping Motivation	Read this as an occlusion decision. Decide whether the method works in object space, image space, or per-fragment depth space.
85	Environment Mapping Process	Read this as sampled data. Identify the texture coordinates, sampling rule, filtering mode, and shader interpretation.
86	Environment Mapping Assumptions	Read this as sampled data. Identify the texture coordinates, sampling rule, filtering mode, and shader interpretation.
87	Environment Mapping 2/2	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
88	Demo - Cube-map Reflection Vector	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
89	Environment Mapping Types 1/2	Read this as sampled data. Identify the texture coordinates, sampling rule, filtering mode, and shader interpretation.
90	Environment Mapping Types 2/2	Read this as sampled data. Identify the texture coordinates, sampling rule, filtering mode, and shader interpretation.
91	Spherical Mapping 1/2	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
92	Spherical Mapping 2/2	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
93	Spherical Environment Mapping	Read this as sampled data. Identify the texture coordinates, sampling rule, filtering mode, and shader interpretation.
94	Sphere Map Distortion	Read this as sampled data. Identify the texture coordinates, sampling rule, filtering mode, and shader interpretation.
95	Demo - Environment Map Distortion	Read this as sampled data. Identify the texture coordinates, sampling rule, filtering mode, and shader interpretation.
96	Sphere Map Interpolation	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
97	Sphere Map Coordinates	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
98	Cube Environment Mapping	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.

Page	Slide title	How to read it
99	Cube Mapping Example 1/2	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
100	Cube Mapping Example 2/2	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
101	Cube Mapping Distortion	Read this as sampled data. Identify the texture coordinates, sampling rule, filtering mode, and shader interpretation.
102	Cube Map Texture Coordinates	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
103	Environment Mapping in OpenGL	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
104	Cube Map Texture Object	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
105	Sampling Direction	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
106	Environment Mapping in GLSL 1/2	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
107	Environment Mapping in GLSL 2/2	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
108	Practical Cube-Map Issues	Read this as sampled data. Identify the texture coordinates, sampling rule, filtering mode, and shader interpretation.
109	<code>1 glEnable(GL_TEXTURE_CUBE_MAP_SEAMLESS);</code>	Read this as sampled data. Identify the texture coordinates, sampling rule, filtering mode, and shader interpretation.
110	Cube Mapping Pros and Cons	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
111	Spherical Environment Mapping Assumptions	Read this as sampled data. Identify the texture coordinates, sampling rule, filtering mode, and shader interpretation.
112	Spherical Environment Mapping Conclusions	Read this as sampled data. Identify the texture coordinates, sampling rule, filtering mode, and shader interpretation.
113	9.7 3D Textures and Volume Rendering	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.

Page	Slide title	How to read it
114	3D Texturing: Relevant Topics	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
115	3D Texture as a Voxel Grid	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
116	Object Space to Texture Space	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
117	3D Texture Access	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
118	Trilinear Interpolation	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
119	$f = (1 - \alpha) \cdot f + \alpha \cdot f$	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
120	3D Texture Memory Footprint	Read this as sampled data. Identify the texture coordinates, sampling rule, filtering mode, and shader interpretation.
121	3D Mipmaps and Filtering	Read this as an occlusion decision. Decide whether the method works in object space, image space, or per-fragment depth space.
122	Solid Texturing	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
123	Volumetric Data as Texture Data	Read this as sampled data. Identify the texture coordinates, sampling rule, filtering mode, and shader interpretation.
124	Transfer Functions	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
125	Volume Rendering with 3D Textures	Read this as sampled data. Identify the texture coordinates, sampling rule, filtering mode, and shader interpretation.
126	Front-to-Back Alpha Compositing	Read this as an occlusion decision. Decide whether the method works in object space, image space, or per-fragment depth space.
127	Ray Marching Volume Rendering	Read this as an occlusion decision. Decide whether the method works in object space, image space, or per-fragment depth space.
128	Gradients and Volume Lighting	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.

Page	Slide title	How to read it
129	Quality and Performance Trade-Offs	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
130	3D Texturing Takeaways	Read this as an occlusion decision. Decide whether the method works in object space, image space, or per-fragment depth space.
131	■ Textures are sampled GPU data sources used by shaders	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
132	Literature and other sources used in this chapter	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
133	■ Text Books	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?

09.1 Texture Objects, Texels, Formats, and Color Space

Source location: Section 9.1

Commentary

A texture object stores sampled data on the GPU. A texel is one stored texture sample. The texture format decides what channels and numeric representation the samples use. Color space matters because color values may be stored nonlinearly, especially for sRGB images.

The key shift is to stop thinking of texture as only a picture. Textures can store color, normals, depth, lookup data, environment information, or volume data. The shader decides how the sampled values are interpreted.

Mental model

A texture is a GPU-accessible sampled data field.

Check yourself

Why can the same texture mechanism store both color maps and normal maps?

09.2 Texture Coordinates, UV Mapping, and Wrapping

Source location: Section 9.2

Commentary

Texture coordinates map surface points to locations in texture space. UV coordinates are usually interpolated across primitives during rasterization and then used by the fragment shader to sample a texture.

Wrapping rules decide what happens outside the normal coordinate range. Repeat, clamp, mirrored repeat, and border behavior are not visual afterthoughts; they define the sampling function outside the base domain.

Mental model

UVs are the address system that lets a fragment look up texture data.

Check yourself

What artifact might appear if a wrapping mode is wrong at the edge of a surface?

09.3 Sampling, Filtering, Mipmaps, and Anisotropy

Source location: Section 9.3

Commentary

Sampling turns continuous texture coordinates into values from discrete texels. Nearest filtering selects a nearby texel and can look blocky. Linear filtering blends nearby texels and looks smoother. Minification is harder because many texels may map to one pixel.

Mipmaps store prefiltered lower-resolution versions of a texture. They reduce aliasing and shimmer when textures are seen at small scale. Anisotropic filtering improves quality when a texture is viewed at a steep angle where footprint shape is elongated.

Mental model

Filtering reconstructs values; mipmaps choose an appropriate scale before reconstruction.

Check yourself

Why does a distant checkerboard shimmer without mipmapping?

09.4 Modern OpenGL Texture Pipeline

Source location: Section 9.4

Commentary

In modern OpenGL, textures are represented by texture objects, bound to texture units, connected to sampler uniforms, and accessed in shaders. The shader does not sample a filename; it samples a bound texture through a sampler.

Texture bugs often come from state mismatches: wrong active texture unit, wrong sampler uniform, missing mipmaps for a mipmap filter, wrong wrap mode, or wrong internal format.

Mental model

OpenGL texturing is a chain: object data -> texture unit -> sampler uniform -> shader lookup.

Check yourself

Why can a texture appear black when the shader code is mathematically correct?

09.5 Normal Mapping

Source location: Section 9.5

Commentary

Normal mapping stores normal directions in a texture so that lighting can vary at a finer scale than the geometry. The surface may have few triangles, but the shader receives detailed normals per fragment.

The hard part is coordinate space. Normal maps are often defined in tangent space, so the shader must transform or interpret them with the correct tangent, bitangent, and normal basis.

Mental model

Normal mapping changes the lighting normal, not the actual mesh silhouette.

Check yourself

Why does normal mapping not change the geometric outline of an object?

09.6 Environment Mapping and 3D Textures

Source location: Sections 9.6-9.7

Commentary

Environment mapping uses textures to represent surrounding illumination or reflections. A direction, not a surface UV alone, can be used to sample an environment map. This is a texture lookup driven by view/reflection geometry.

3D textures and volume rendering extend the same sampled-data idea into three dimensions. Instead of sampling a 2D image, the shader samples a volume. This is useful for data such as medical scans, density fields, or procedural volumetric effects.

Mental model

Textures are general sampled data; the coordinate dimensionality depends on the problem.

Check yourself

What coordinate type would you use to sample a 3D texture?

End-of-lecture summary

If you remember only one thing from Lecture 09, remember this: This lecture treats textures as sampled data for shading, not merely images glued onto objects. It connects UV coordinates, sampling, filtering, mipmaps, OpenGL texture state, and advanced texture uses.

Before moving on, you should be able to explain the lecture without reading slide bullets aloud. Use the vocabulary from the slides, but add causal links: why a step exists, what data it consumes, what it produces, and what can go wrong.

Lecture 10 - Shadows

This lecture explains shadows as a visibility problem from the light source. A point is lit if the light can see it; it is shadowed if another object blocks that visibility.

Source chunk	course_text_parts/03_lectures/10_shadows.txt
Extracted slide pages	60
Reader purpose	Commentary and causal explanation for human reading

How to read this lecture

- Read the section commentary before looking at the raw slide bullets.
- Use the mental model as the short version you should remember.
- Answer the check question without notes before continuing.

Slide walkthrough

Each row gives a short reading comment for one extracted slide page. Use it to turn sparse slide bullets into a human-readable path through the lecture.

Page	Slide title	How to read it
1	Untitled slide	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
2	Shadow Perception 1/3	Read this as an occlusion decision. Decide whether the method works in object space, image space, or per-fragment depth space.
3	Shadow Perception 2/3	Read this as light-space visibility. Ask what blocks the light, what representation stores that information, and which artifact can appear.
4	Shadow Perception 3/3	Read this as light-space visibility. Ask what blocks the light, what representation stores that information, and which artifact can appear.
5	Shadow Atmosphere	Read this as light-space visibility. Ask what blocks the light, what representation stores that information, and which artifact can appear.
6	■ Shadows create a certain atmosphere	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
7	Shadow Atmosphere	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
8	Overview	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.

Page	Slide title	How to read it
9	10.1 Definitions	Read this as light-space visibility. Ask what blocks the light, what representation stores that information, and which artifact can appear.
10	Classification of scene objects	Read this as light-space visibility. Ask what blocks the light, what representation stores that information, and which artifact can appear.
11	Umbra and Penumbra	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
12	Hard and Soft Shadows	Read this as light-space visibility. Ask what blocks the light, what representation stores that information, and which artifact can appear.
13	Description of Shadows	Read this as light-space visibility. Ask what blocks the light, what representation stores that information, and which artifact can appear.
14	■ Shadows can be described differently	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
15	Observations 1/2	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
16	Observations 2/2	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
17	Shadow Calculation Methods	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
18	10.2 Ground Plane Shadows	Read this as light-space visibility. Ask what blocks the light, what representation stores that information, and which artifact can appear.
19	Static Shadow Polygons	Read this as an occlusion decision. Decide whether the method works in object space, image space, or per-fragment depth space.
20	Render Static Shadow Polygons 1/2	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
21	Render Static Shadow Polygons 2/2	Read this as an occlusion decision. Decide whether the method works in object space, image space, or per-fragment depth space.
22	Static Shadow Polygons	Read this as light-space visibility. Ask what blocks the light, what representation stores that information, and which artifact can appear.
23	Dynamic Shadow Polygons	Read this as light-space visibility. Ask what blocks the light, what representation stores that information, and which artifact can appear.

Page	Slide title	How to read it
24	Shadow Projection	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
25	Rendering Dynamic Shadow Polygons	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
26	10.3 Light Maps	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
27	Light Maps	Read this as an occlusion decision. Decide whether the method works in object space, image space, or per-fragment depth space.
28	Soft Shadows with Light Maps 1/2	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
29	Soft Shadows with Light Maps 2/2	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
30	10.4 Shadow Volumes	Read this as light-space visibility. Ask what blocks the light, what representation stores that information, and which artifact can appear.
31	Shadow Volumes	Read this as light-space visibility. Ask what blocks the light, what representation stores that information, and which artifact can appear.
32	Initial Considerations	Read this as an occlusion decision. Decide whether the method works in object space, image space, or per-fragment depth space.
33	Shadow Volume Geometry	Read this as light-space visibility. Ask what blocks the light, what representation stores that information, and which artifact can appear.
34	Shadow Volume Realization	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
35	Shadow Volume Properties 1/2	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
36	Shadow Volume Properties 2/2	Read this as light-space visibility. Ask what blocks the light, what representation stores that information, and which artifact can appear.
37	Stencil Shadow Volume	Read this as light-space visibility. Ask what blocks the light, what representation stores that information, and which artifact can appear.
38	Implementation	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.

Page	Slide title	How to read it
39	Special Case: COP in the shadows	Read this as light-space visibility. Ask what blocks the light, what representation stores that information, and which artifact can appear.
40	Shadow Volume Example	Read this as light-space visibility. Ask what blocks the light, what representation stores that information, and which artifact can appear.
41	Shadow Volume Conclusions	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
42	10.5 Shadow Maps	Read this as an occlusion decision. Decide whether the method works in object space, image space, or per-fragment depth space.
43	Shadow Mapping	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
44	Projected Shadows	Read this as an occlusion decision. Decide whether the method works in object space, image space, or per-fragment depth space.
45	Untitled slide	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
46	Depth Comparison	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
47	Coordinate Systems	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
48	Pseudo Code	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
49	Properties	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
50	View Volume Problem	Read this as an occlusion decision. Decide whether the method works in object space, image space, or per-fragment depth space.
51	Bias Problem 1/2	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
52	Bias Problem 2/2	Read this as light-space visibility. Ask what blocks the light, what representation stores that information, and which artifact can appear.
53	Aliasing Problem 1/3	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.

Page	Slide title	How to read it
54	Aliasing Problem 2/3	Read this as an occlusion decision. Decide whether the method works in object space, image space, or per-fragment depth space.
55	Untitled slide	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
56	Aliasing Problem 3/3	Read this as sampled data. Identify the texture coordinates, sampling rule, filtering mode, and shader interpretation.
57	Untitled slide	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
58	■ Shadows in Visual Perception	Read this as an occlusion decision. Decide whether the method works in object space, image space, or per-fragment depth space.
59	Literature and other sources used in this chapter	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
60	■ Text Books	Read this as light-space visibility. Ask what blocks the light, what representation stores that information, and which artifact can appear.

10.1 Definitions

Source location: Section 10.1

Commentary

A shadow is not simply a dark texture. It is evidence that light visibility is blocked. Important distinctions include hard versus soft shadows, umbra versus penumbra, and geometric versus precomputed approaches.

For real-time graphics, shadows are usually approximated. The approximation must answer a visibility question cheaply enough for interactive rendering.

Mental model

Shadows are light-space visibility made visible in the camera image.

Check yourself

Why is shadow computation related to visibility determination?

10.2 Ground Plane Shadows

Source location: Section 10.2

Commentary

Ground plane shadows project an object onto a receiving plane. This is simple and can look convincing in restricted scenes, but it assumes a known planar receiver and does not handle general shadowing between

arbitrary objects.

The method is useful pedagogically because it makes the geometric nature of shadows explicit: a light source, an occluder, and a receiver define where the shadow lands.

Mental model

A planar shadow is projected geometry on a known receiver.

Check yourself

What scene limitation makes ground plane shadows less general than shadow maps?

10.3 Light Maps

Source location: Section 10.3

Commentary

Light maps store precomputed lighting or shadow information in textures. They can be efficient at runtime, but they are limited when lights, objects, or geometry move dynamically.

This is another example of a recurring graphics tradeoff: precompute for speed, but lose flexibility. Real-time rendering often mixes precomputed and dynamic techniques.

Mental model

A light map spends storage and preprocessing to save runtime shading work.

Check yourself

Why are light maps less suitable for fully dynamic moving lights?

10.4 Shadow Volumes

Source location: Section 10.4

Commentary

Shadow volumes construct the volume of space blocked from a light by an occluder. The camera view can then determine whether visible points lie inside that volume. Stencil buffer techniques are often associated with this method.

The strength is geometric precision for hard shadows. The cost is handling silhouette edges, volume construction, and robust stencil operations. It is a good contrast to shadow maps, which are image-based from the light view.

Mental model

A shadow volume is the 3D region where the light cannot reach.

Check yourself

Why are silhouette edges important for constructing shadow volumes?

10.5 Shadow Maps

Commentary

Shadow mapping renders the scene from the light's point of view and stores depth. During the camera pass, a surface point is transformed into light space and compared against the stored depth. If it is farther than what the light saw, it is in shadow.

This method is practical and widely used, but it has artifacts. Shadow acne comes from precision/self-comparison issues. Bias can reduce acne but too much bias causes detached shadows. Resolution, filtering, and light projection strongly affect quality.

Mental model

Shadow maps reuse the depth-buffer idea, but from the light's camera.

Check yourself

What exactly is stored in a shadow map?

End-of-lecture summary

If you remember only one thing from Lecture 10, remember this: This lecture explains shadows as a visibility problem from the light source. A point is lit if the light can see it; it is shadowed if another object blocks that visibility.

Before moving on, you should be able to explain the lecture without reading slide bullets aloud. Use the vocabulary from the slides, but add causal links: why a step exists, what data it consumes, what it produces, and what can go wrong.